

ITA06 - Machine Learning

Exp 11 : Write a program for the task of Credit Score Classification

Aim

To implement Credit Score Classification using the Decision Tree Classification Algorithm on different customer financial datasets and predict the credit score category (Excellent, Good, Average, Poor).

Software Requirements

- Python 3.x
- Google Colab / Jupyter Notebook
- Pandas
- NumPy
- Scikit-learn

Algorithm

1. Import the required Python libraries.
2. Create the dataset using Pandas DataFrame.
3. Convert categorical values into numerical values using Label Encoder.
4. Separate input features and target labels.
5. Train the Decision Tree Classifier.
6. Provide test data for prediction.
7. Predict the Credit Score category.
8. Display the classification result.

Dataset 1: Customer Loan Profile

Annual Income (₹ Lakhs)	Credit History (Years)	Existing Loans	EMI Paid on Time	Credit Score
12	10	1	Yes	Excellent
10	8	2	Yes	Good
8	6	2	Yes	Good
6	4	3	No	Average

Annual Income (₹ Lakhs)	Credit History (Years)	Existing Loans	EMI Paid on Time	Credit Score
4	2	4	No	Poor
15	12	1	Yes	Excellent
7	5	3	Yes	Average
5	3	4	No	Poor
11	9	2	Yes	Good
3	1	5	No	Poor

Target Variable

Credit Score: Excellent, Good, Average, Poor

New Sample

Annual Income Credit History Existing Loans EMI Paid on Time

9 7 2 Yes

Dataset 2: Bank Customer Credit Rating

Monthly Income (₹) Age Loan Amount (₹ Lakhs) Loan Default Credit Score

90000	45	5	No	Excellent
75000	40	6	No	Good
60000	35	8	No	Good
45000	30	10	Yes	Average
30000	25	12	Yes	Poor
95000	48	4	No	Excellent
65000	37	7	No	Good
40000	28	9	Yes	Average
28000	24	13	Yes	Poor

Monthly Income (₹) Age Loan Amount (₹ Lakhs) Loan Default Credit Score

85000 43 5 No Excellent

Target Variable

Credit Score

New Sample

Monthly Income Age Loan Amount Loan Default

70000 38 6 No

Dataset 3: Credit Card Customer Analysis

Credit Limit (₹ Lakhs) Monthly Spending (₹) Payment Delay (Days) Credit Score

8 40000 0 Excellent

7 35000 2 Good

6 30000 4 Good

5 25000 10 Average

4 20000 20 Poor

9 45000 0 Excellent

6 28000 5 Good

5 23000 12 Average

3 18000 25 Poor

8 42000 1 Excellent

Target Variable

Credit Score

New Sample

Credit Limit Monthly Spending Payment Delay

7 36000 3

Dataset 4: Personal Finance Profile

Savings (₹ Lakhs) Monthly Expenses (₹) Outstanding Debt (₹ Lakhs) Credit Score

10	30000	1	Excellent
8	35000	2	Good
6	40000	3	Good
4	45000	5	Average
2	50000	7	Poor
11	28000	1	Excellent
7	36000	2	Good
5	43000	4	Average
3	48000	6	Poor
9	32000	1	Excellent

Target Variable

Credit Score

New Sample

Savings Monthly Expenses Outstanding Debt

7 34000 2

Dataset 5: Home Loan Eligibility

Salary (₹ Lakhs) CIBIL Score Existing EMIs Loan Status Credit Score

15	850	1	Approved	Excellent
12	800	2	Approved	Good
10	760	2	Approved	Good
8	700	3	Pending	Average
6	620	4	Rejected	Poor
16	880	1	Approved	Excellent
11	780	2	Approved	Good
7	680	3	Pending	Average
5	600	5	Rejected	Poor
14	830	1	Approved	Excellent

Target Variable

Credit Score

New Sample

Salary CIBIL Score Existing EMIs Loan Status

11 790 2 Approved

[1] ✓ 2s (Ctrl+Enter)
 CPython 3.12.2 (tags/v3.12.2:6abdd70, Oct 6, 2023)
 Executed since last change: 2 minutes ago
 Completed in 2.27 s

```

import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = {
    'Annual_Income': [12,10,8,6,4,15,7,5,11,3],
    'Credit_History': [10,8,6,4,2,12,5,3,9,1],
    'Existing_Loans': [1,2,2,3,4,1,3,4,2,5],
    'EMI_Paid_On_Time': [1,1,1,0,0,1,1,0,1,0],
    'Credit_Score': [3,2,2,1,0,3,1,0,2,0]
}

df = pd.DataFrame(data)

# Features and Target
X = df[['Annual_Income', 'Credit_History',
        'Existing_Loans', 'EMI_Paid_On_Time']]
y = df['Credit_Score']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# Train Decision Tree
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

```

```

print("Predicted Credit Score:",
      labels[prediction[0]])

```

```

... Accuracy: 0.6666666666666666
Predicted Credit Score: Good
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have
warnings.warn(

```

[2]
✓ Os

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Dataset
data = {
    'Annual_Income': [12, 10, 8, 6, 4, 15, 9, 5, 3, 13],
    'Credit_History': [10, 8, 6, 4, 2, 12, 7, 3, 1, 11],
    'Existing_Loans': [1, 2, 2, 3, 4, 1, 2, 3, 5, 1],
    'EMI_Paid_On_Time': ['Yes', 'Yes', 'Yes', 'No', 'No',
                        'Yes', 'Yes', 'No', 'No', 'Yes'],
    'Credit_Score': ['Excellent', 'Good', 'Good', 'Average', 'Poor',
                    'Excellent', 'Good', 'Average', 'Poor', 'Excellent']
}

df = pd.DataFrame(data)

# Convert categorical data into numeric form
le_emi = LabelEncoder()
df['EMI_Paid_On_Time'] = le_emi.fit_transform(df['EMI_Paid_On_Time'])

le_score = LabelEncoder()
df['Credit_Score'] = le_score.fit_transform(df['Credit_Score'])

# Features and Target
X = df[['Annual_Income', 'Credit_History',
        'Existing_Loans', 'EMI_Paid_On_Time']]
y = df['Credit_Score']

# Train Decision Tree Classifier
model = DecisionTreeClassifier()
```

```
# Features and Target
X = df[['Annual_Income', 'Credit_History',
        'Existing_Loans', 'EMI_Paid_On_Time']]
y = df['Credit_Score']

# Train Decision Tree Classifier
model = DecisionTreeClassifier()
model.fit(X, y)

# Test Data
test = [[11, 9, 1, 1]] # Income=11, History=9, Loans=1, EMI=Yes

# Prediction
prediction = model.predict(test)

print("Predicted Credit Score:",
      le_score.inverse_transform(prediction)[0])
```

```
... Predicted Credit Score: Excellent
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have v
warnings.warn(
```

[3]
✓ Os

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Dataset
data = {
    'Credit_Limit': [8, 7, 6, 5, 4, 9, 6, 5, 3, 8],
    'Monthly_Spending': [40000, 35000, 30000, 25000, 20000,
                        45000, 28000, 23000, 18000, 42000],
    'Payment_Delay': [0, 2, 4, 10, 20, 0, 5, 12, 25, 1],
    'Credit_Score': ['Excellent', 'Good', 'Good', 'Average', 'Poor',
                    'Excellent', 'Good', 'Average', 'Poor', 'Excellent']
}

df = pd.DataFrame(data)

# Encode target labels
le = LabelEncoder()
df['Credit_Score'] = le.fit_transform(df['Credit_Score'])

# Features and Target
X = df[['Credit_Limit', 'Monthly_Spending', 'Payment_Delay']]
y = df['Credit_Score']

# Train Decision Tree Classifier
model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)

# Test Customer
test_customer = [[7, 38000, 2]]

# Prediction
```

```
# Test Customer
test_customer = [[7, 38000, 2]]

# Prediction
prediction = model.predict(test_customer)

print("Predicted Credit Score:",
      le.inverse_transform(prediction)[0])
```

```
... Predicted Credit Score: Excellent
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have v
warnings.warn(
```


[4]
✓ Os

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Dataset
data = {
    'Savings': [10, 8, 6, 4, 2, 11, 7, 5, 3, 9],
    'Monthly_Expenses': [30000, 35000, 40000, 45000, 50000,
                        28000, 36000, 43000, 48000, 32000],
    'Outstanding_Debt': [1, 2, 3, 5, 7, 1, 2, 4, 6, 1],
    'Credit_Score': ['Excellent', 'Good', 'Good', 'Average', 'Poor',
                    'Excellent', 'Good', 'Average', 'Poor', 'Excellent']
}

df = pd.DataFrame(data)

# Encode Credit Score labels
le = LabelEncoder()
df['Credit_Score'] = le.fit_transform(df['Credit_Score'])

# Features and Target
X = df[['Savings', 'Monthly_Expenses', 'Outstanding_Debt']]
y = df['Credit_Score']

# Train Decision Tree Classifier
model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)

# Test Customer
test_customer = [[8, 34000, 2]]
```

V

```
# Train Decision Tree Classifier
model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)

# Test Customer
test_customer = [[8, 34000, 2]]

# Prediction
prediction = model.predict(test_customer)

print("Predicted Credit Score:",
      le.inverse_transform(prediction)[0])
```

```
... Predicted Credit Score: Good
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have v
warnings.warn(
```

[5]
✓ Os

d and replace

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder

# Dataset
data = {
    'Salary': [15, 12, 10, 8, 6, 16, 11, 7, 5, 14],
    'CIBIL_Score': [850, 800, 760, 700, 620, 880, 780, 680, 600, 830],
    'Existing_EMIs': [1, 2, 2, 3, 4, 1, 2, 3, 5, 1],
    'Loan_Status': ['Approved', 'Approved', 'Approved', 'Pending', 'Rejected',
                    'Approved', 'Approved', 'Pending', 'Rejected', 'Approved'],
    'Credit_Score': ['Excellent', 'Good', 'Good', 'Average', 'Poor',
                    'Excellent', 'Good', 'Average', 'Poor', 'Excellent']
}

df = pd.DataFrame(data)

# Convert categorical column Loan_Status into numeric values
le_status = LabelEncoder()
df['Loan_Status'] = le_status.fit_transform(df['Loan_Status'])

# Convert Credit_Score into numeric values
le_score = LabelEncoder()
df['Credit_Score'] = le_score.fit_transform(df['Credit_Score'])

# Features and Target
X = df[['Salary', 'CIBIL_Score', 'Existing_EMIs', 'Loan_Status']]
y = df['Credit_Score']

# Train Decision Tree Classifier
model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)
```

```
# Convert Credit_Score into numeric values
le_score = LabelEncoder()
df['Credit_Score'] = le_score.fit_transform(df['Credit_Score'])

# Features and Target
X = df[['Salary', 'CIBIL_Score', 'Existing_EMIs', 'Loan_Status']]
y = df['Credit_Score']

# Train Decision Tree Classifier
model = DecisionTreeClassifier(random_state=0)
model.fit(X, y)

# Test Customer
# Salary=13 Lakhs, CIBIL=810, EMIs=2, Loan Status=Approved
test_customer = [[13, 810, 2, le_status.transform(['Approved'])[0]]]

# Prediction
prediction = model.predict(test_customer)

print("Predicted Credit Score:",
      le_score.inverse_transform(prediction)[0])
```

```
... Predicted Credit Score: Good
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have v
warnings.warn(
```

Result

Thus, the Credit Score Classification model was successfully implemented using the Decision Tree Classification Algorithm. The model was trained on customer financial datasets and accurately classified customers into Excellent, Good, Average, and Poor credit score categories based on their financial profiles, loan history, repayment behavior, and credit-related attributes. The predicted results demonstrate that machine learning can effectively assist financial institutions in assessing customer creditworthiness and supporting loan approval decisions.